

```
l=[]  
print(l)
```

↔ []

```
l=list()  
print(l)
```

↔ []

```
l=list("1","2")  
print(l)
```

↔

```
-----  
TypeError                                Traceback (most recent call last)  
<ipython-input-6-efe6ac2ba761> in <cell line: 1>()  
----> 1 l=list("1","2")  
      2 print(l)  
  
TypeError: list expected at most 1 argument, got 2
```

```
l=list(("1","2")) #inside list(),elements should be given in ()  
print(l)
```

↔ ['1', '2']

```
print(l[0])
```

↔ 1

```
t=tuple(("3",4))  
l=list(t)  
print(l)
```

↔ ['3', 4]

```
str1="Ashika Manohar"  
print(list(str1))
```

↔ ['A', 's', 'h', 'i', 'k', 'a', ' ', 'M', 'a', 'n', 'o', 'h', 'a', 'r']

```
l = [1,2,3,"a","b","c","rcb","gate","da"]  
print(l[4])  
print(l[-4])
```

↔ b
c

```
l = [[1,2,3,[4,5]],[23,[12,34],9],30,21]  
print(len(l))
```

↔ 4

```
print(l[0][2])
```

↔ 3

```
print(l[0][3])
```

↔ [4, 5]

```
print(l[0][3][1])
```

↔ 5

```
alpha = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j']
#               -4   8
print(alpha[-4:8])
print(alpha[-4:-8]) #it is adding 1 after 1 to get -8 which is not possible..so empty list
```

```
↔ ['g', 'h']
[]
```

```
l = ["sachin", "sehwag", "gauti", "virat", "yuvi", "raina", "dhoni", "yousuf", "zaheer", "bumrah"]
#               2               -4
print(l[-4:2]) #so won't print anything....
```

```
↔ []
```

```
l = ["sachin", "sehwag", "gauti", "virat", "yuvi", "raina", "dhoni", "yousuf", "zaheer", "bumrah"]
#               2               by -1<--- 6
l[6:2:-1]
```

```
↔ ['dhoni', 'raina', 'yuvi', 'virat']
```

```
l[::-1] #reverse a list
```

```
↔ ['bumrah',
    'zaheer',
    'yousuf',
    'dhoni',
    'raina',
    'yuvi',
    'virat',
    'gauti',
    'sehwag',
    'sachin']
```

```
#adding elements
```

```
csk = ["Hayden", "Vijay", "Raina"]
print(csk)
print(id(csk))
csk.append("Dhoni")
print(csk)
print(id(csk)) #id won't change when we add new elements
```

```
↔ ['Hayden', 'Vijay', 'Raina']
134105000882560
['Hayden', 'Vijay', 'Raina', 'Dhoni']
134105000882560
```

```
l=[2,5,4]
l.append("a","b")
```

```
↔ -----
TypeError                                Traceback (most recent call last)
<ipython-input-22-f87f36602057> in <cell line: 2>()
      1 l=[2,5,4]
----> 2 l.append("a","b")

TypeError: list.append() takes exactly one argument (2 given)
```

```
l=[2,5,4]
l.append(["a","b"]) #l.append return none
print(l)
```

```
↔ [2, 5, 4, ['a', 'b']]
```

```
print(l.append("g"))
```

```
↔ None
```

```
l=[2,5,4]
n=l.append(["a","b"])
print(n)
```

➦ None

```
#insert method          list.insert(pos, elmnt)
```

```
indian_batting = ["Rohit", "Virat", "PANT","SKY"]
```

```
indian_batting.insert(1,"Raina")
print(indian_batting)
```

➦ ['Rohit', 'Raina', 'Virat', 'PANT', 'SKY']

```
indian_batting[0]="jadeja"    #by doing this rohit is gone....it is being replaced
print(indian_batting)
```

➦ ['jadeja', 'Raina', 'Virat', 'PANT', 'SKY']

```
l=indian_batting.insert(5,"Sachin")  #added in the last
print(l)
print(indian_batting)
```

➦ None
['jadeja', 'Raina', 'Virat', 'PANT', 'SKY', 'Sachin']

```
indian_batting.insert(8,"Raina")  #even given index 8...no element in 6 7,so it gets printed as 6th one
print(indian_batting)
```

➦ ['jadeja', 'Raina', 'Virat', 'PANT', 'SKY', 'Sachin', 'Raina']

```
l = [1,2,3,4,5,6]
l.insert(-2, 'abc')
print(l)
```

➦ [1, 2, 3, 4, 'abc', 5, 6]

```
l = [1,2,3,4,5]
l.insert(2, [10,20,30])
print(l)
```

➦ [1, 2, [10, 20, 30], 3, 4, 5]

```
#extend method --To append elements from another list to the current list, use the extend() method.
sub=["cn","os","dbms"]
print(sub)
s=["cd","ld","maths"]
print(sub.extend(s))
print(s.extend(sub))
```

➦ ['cn', 'os', 'dbms']
None
None

```
sub=["cn","os","dbms"]
print(sub)
s=["cd","ld","maths"]
sub.extend(s)
print(sub)
s.extend(sub)  #now sub is ['cn', 'os', 'dbms', 'cd', 'ld', 'maths']
print(s)
```

➦ ['cn', 'os', 'dbms']
['cn', 'os', 'dbms', 'cd', 'ld', 'maths']
['cd', 'ld', 'maths', 'cn', 'os', 'dbms', 'cd', 'ld', 'maths']

```
a=["s","f","e",3]
a.append("[2,4]") #here it is added as a list
print(a)
a=["s","f","e",3]
a.extend([20,30]) #here it is printed not as a list...but as individual elements
print(a)
```

```
→ ['s', 'f', 'e', 3, '[2,4]']
   ['s', 'f', 'e', 3, 20, 30]
```

```
b=[3,5,9]
b.append("abc")
print(b)
b=[3,5,9]
b.extend("abc") #now here when using extend,it expands the whole str and add each char as an element
print(b)
```

```
→ [3, 5, 9, 'abc']
   [3, 5, 9, 'a', 'b', 'c']
```

```
#user input
```

```
n=input().split()
print(n)
```

```
→ 1 2 3 4
   ['1', '2', '3', '4']
```

```
l=[]
n=int(input("Enter no of items:"))
for i in range(n):
    l.append(input(f"Enter element{i}"))
print(l)
```

```
→ Enter no of items:5
   Enter element090
   Enter element120
   Enter element230
   Enter element324
   Enter element433
   ['90', '20', '30', '24', '33']
```

```
players = ["Rohit", "Jaiswal", "Virat", "Pant", "Sky", "Dube", "Hardik", "Jaddu"]
print(players)
players[5] = "Rinku"
print(players)
```

```
→ ['Rohit', 'Jaiswal', 'Virat', 'Pant', 'Sky', 'Dube', 'Hardik', 'Jaddu']
   ['Rohit', 'Jaiswal', 'Virat', 'Pant', 'Sky', 'Rinku', 'Hardik', 'Jaddu']
```

```
players[5:]="Rinku"
print(players)
```

```
→ ['Rohit', 'Jaiswal', 'Virat', 'Pant', 'Sky', 'R', 'i', 'n', 'k', 'u']
```

```
players[7:9]="12"
print(players)
```

```
→ ['Rohit', 'Jaiswal', 'Virat', 'Pant', 'Sky', 'R', 'i', '1', '2', 'u']
```

```
players = ["Rohit", "Jaiswal", "Virat", "Pant", "Sky", "Dube", "Hardik", "Jaddu"]
print(players)
players[5:] = ["Rinku"] #now it won't print each char...print as 1 element
print(players)
```

```
→ ['Rohit', 'Jaiswal', 'Virat', 'Pant', 'Sky', 'Dube', 'Hardik', 'Jaddu']
   ['Rohit', 'Jaiswal', 'Virat', 'Pant', 'Sky', 'Rinku']
```

```
players = ["Rohit", "Jaiswal", "Virat", "Pant", "Sky", "Dube", "Hardik", "Jaddu"]
print(players)
players[5:7] = ["Rinku", "Tewatia", "Ashuthosh", "Shashank"]
print(players)
```

```
['Rohit', 'Jaiswal', 'Virat', 'Pant', 'Sky', 'Dube', 'Hardik', 'Jaddu']
['Rohit', 'Jaiswal', 'Virat', 'Pant', 'Sky', 'Rinku', 'Tewatia', 'Ashuthosh', 'Shashank', 'Jaddu']
```

```
#remove
l=[4,5,6,8]
a=l.remove(5)
print(l)
print(a)
```

```
[4, 6, 8]
None
```

```
l=[4,2,3,2,3,9,2]
l.remove(2) #remove only the 1st 2..others 2 are preserved
print(l)
```

```
[4, 3, 2, 3, 9, 2]
```

```
#del
l=[4,68,3,4,8]
del l[4]
print(l)
```

```
[4, 68, 3, 4]
```

```
l = [1,2,3,4,5]
print(l)
l.clear()
print(l)
```

```
[1, 2, 3, 4, 5]
[]
```

```
#pop
l = [1,2,3,4,5] #pop the element in the index specified...if not specified then pop last element in the list
print(l)
a = l.pop(3)
print(l)
print(a)
```

```
[1, 2, 3, 4, 5]
[1, 2, 3, 5]
4
```

```
l = [1,2,3,4,5]
print(l)
a = l.pop(10)
print(l)
print(a)
```

```
[1, 2, 3, 4, 5]
```

```
-----
IndexError                                Traceback (most recent call last)
<ipython-input-3-67f44e0a84d1> in <cell line: 3>()
      1 l = [1,2,3,4,5]
      2 print(l)
----> 3 a = l.pop(10)
      4 print(l)
      5 print(a)

IndexError: pop index out of range
```

Next steps: [Explain error](#)

→ [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36,

⇒ [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36,

→ [2, 4, 6, 8, 10]

➡ [11, 7, 13, 9, 15, 11, 17, 13, 19]

→ [(1, 3), (1, 4), (2, 2), (2, 3), (2, 4), (3, 2), (3, 3), (3, 4)]

✓ Coding questions practice









→ [2, 3, 4, 5]

```
[>] ['a', 's', 'h', 'i', 'k', 'a', ' ', 'm', 'a', 'n', 'o', 'h', 'a', 'r']
```

```
l=[" 223","sdbj"]
print(" ".join(l))
```

→ 223 sdbj

```
s="weer wdekf 23 33ef"
print(s.split(" "))
```

→ ['weer', 'wdekf', '', '23', '33ef']

```
l=6
st=6
print(f"Stuart {st+l}")
```

→ Stuart 12

```
l=["sdd","ddv",2,2,4,5,6]

print(l[::-1])
```

→ [6, 5, 4, 2, 2, 'ddv', 'sdd']

```
s="sjbsj sdbjsd jbj"
l=s.split()
print(l[::-1])
n=l[::-1]
print(" ".join(n))
```

→ ['jbj', 'sdbjsd', 'sjbsj']
jbj sdbjsd sjbsj

```
stre="aabbss"
print(stre.count("a"))
```

→ 2

```
from collections import Counter
s="abbbsjbj"
n=Counter(s)
print(n)
for i in s:
    if n[i]==1:
        print(i)
        break
    else:
        print("$")
        break
```

→ Counter({'b': 4, 'j': 2, 'a': 1, 's': 1})
a

```
s="abaab"
s=s[-1]+s[:len(s)-1]
print(s)
```

→ babaa

```
s="gvxc"
s=s[-1]+s[:len(s)-1]
s=s[-1]+s[:len(s)-1]
print(s)
```

→ xcv

```
s1="gvxc"
s1=s1[1:]+s1[0]
s1=s1[1:]+s1[0]
print(s1)
```

 xcgv

```
s1="gvxc"
s2="xcgv"
if (s1==s2)==0:
    print("false")
```

 false

False==1

 False

```
s1=input()
s2=input()
r1=s1[2:]+s1[:2]
r2=s1[-2:]+s1[:-2]
if r1==s2 or r2==s2:
    print("true")
else:
    print("false")
```

 gvxc
cgxv
false

```
list1=[3,"sbj","ashi"]
print(reversed(list1))
```

 <list_reverseiterator object at 0x7e12752fc850>

```
list1=[3,"sbj","ashi"]
print(list(reversed(list1)))
```

 ['ashi', 'sbj', 3]

```
s="abdvh dac 30 we8"
print(s.split(" "))
```

 ['abdvh', 'dac', '', '30', 'we8']

```
li=[3,4,2,11,5]
print(sorted(li))
```

 [2, 3, 4, 5, 11]

```
s="abdvh dac 30 we8"
l=s.split(" ")
print(" ".join(l[::-1]))
```

 we8 30 dac abdvh

```
s="abdvh dac 30 we8"
l=s.split(" ")[::-1]
print(" ".join(l))
```

 we8 30 dac abdvh

```
s=input()
l=s.split(" ")
n=[]
for i in l:
    if len(i)%2==0:
        n.append(i)
print(n)
print(" ".join(n))
```



```
↗ this is my life i love it
  ['this', 'is', 'my', 'life', 'love', 'it']
  this is my life love it
```

```
s=input()
mid=len(s)//2
if len(s)%2==0:
    print(s[:mid]==s[mid:])
else:
    print(s[:mid+1]==s[mid:])
```

```
↗ amaama
  True
```

```
s="ganx  ss  n sxx d "
count=0
for i in s:
    if i!=" ":
        count+=1
print(count)
```

```
↗ 11
```

```
s="ganx  ss  n sxx d "
c=0
for i in s:
    if i.isspace():
        continue
    c+=1
print(c)
```

```
↗ 11
```

```
test_str = 'geeksforgeeks'
mid=len(test_str)//2
print(test_str[:mid]+test_str[mid:].upper())
```

```
↗ geeksFORGEEKS
```

```
from collections import Counter
s="ashika"
Counter(s)
```

```
↗ Counter({'a': 3, 's': 1, 'h': 1, 'i': 1, 'k': 1})
```

```
s="sdfds"
s.count("s")
```

```
↗ 2
```

```
#ashika
#21111

s="ASHIKA"
s1=""
l=[]
for i in s:
    if i not in s1:
        s1+=i
        l.append(s.count(i))
print(l)
```

```
↗ [2, 1, 1, 1, 1]
```

```
s="FRUIT"
s1=""
l=[]
for i in s:
```

```
if i not in s1:
    s1+=i
    l.append(str(s.count(i)))
print("".join(l))
```

 11111

```
s="AAA"
s1=""
l=[]
for i in s:
    if i not in s1:
        s1+=i
        l.append(str(s.count(i)))
print("".join(l))
```

 3

```
s="ABCD CD"
s1=""
l=[]
for i in s:
    if i not in s1:
        s1+=i
        l.append(str(s.count(i)))
print("".join(l))
```

 1122

```
s="sdnsj23n %% "
l=[]
for i in s:
    l.append(ord(i))
print(l)
```

 [115, 100, 110, 115, 106, 50, 51, 110, 32, 37, 35, 32, 32]

```
ord("N")
```

 78

```
s="abCDe"
l=[]
for i in s:
    l.append(ord(i))
print(l)
```

 [97, 98, 67, 68, 101]

```
ord("0")
```

 79

```
chr(97)
```

 'a'

```
ord("a")+13
```

 -----
Traceback (most recent call last)
[ipython-input-61-69c4e0bc4d9b](#) in <cell line: 1>()
----> 1 ord("a")+13

TypeError: unsupported operand type(s) for +: 'int' and 'str'

Next steps: [Explain error](#)

```
ord("P")
```

↔ 80

```
s="abCDe"  
s.upper()  
n=""  
for i in s:  
    n+=chr(ord(i)+13)  
print(n.upper())
```

↔ NOPQR

```
s="WxYZ"  
s.upper()  
n=""  
for i in s:  
    if ord(i)<77:  
        n+=chr(ord(i)+13)  
    else:  
        n+=chr(ord(i)-13)  
print(n.upper())
```

↔ JKLM

ord("W")

↔ 87

87-13

↔ 74

chr(74)

↔ 'J'

ord("Z")

↔ 90

89+13

↔ 102

chr(102)

↔ 'f'

90-13

↔ 77

```
s="WxYZ"  
n=""  
for i in s:  
    if ord(i)<77:  
        n+=chr(ord(i)+13)  
    else:  
        n+=chr(ord(i)-13)  
print(n.upper())
```

↔ JKLM

s="abCDe"

```
n=""  
for i in s:  
    if ord(i)<77 or ord(i)>=97:  
        n+=chr(ord(i)+13)  
    else:
```

```
n+=chr(ord(i)-13)
print(n.upper())
```

↔ NOPQR

```
ord("a")
```

↔ 97

```
ord("z")
```

↔ 122

```
ord("A")
```

↔ 65

```
ord("Z")
```

↔ 90

```
s="WxYZ"
```

```
n=""
for i in s:
    if ord(i)<77 or ord(i)>=97:
        n+=chr(ord(i)+13)
    else:
        n+=chr(ord(i)-13)
print(n.upper())
```

↔ JLM

```
ord("x")
```

↔ 120

```
ord("W")
```

↔ 87

```
87+13
```

↔ 100

```
chr(100)
```

↔ 'd'

```
ascii("a")
```

↔ 'a'

Start coding or [generate](#) with AI.

↔ '100'

```
s="WxYZ"
s.upper()
n=""
for i in s:
    n+=chr(ord(i)+13)
print(n.upper())
```

↔ DCFG

```
s="abCDe"
s.upper()
n=""
for i in s:
```

```
n+=chr(ord(i)+13)
print(n.upper())
```

→ NOPQR

```
s="WxYZ"
s.upper()
n=""
for i in s:
    n+=chr(ord(i)+13)
print(n.upper())
```

→ DFG

```
ord("W")
```

→ 87

```
87+13
```

→ 100

```
ord("Z")
```

→ 90

```
ord("a")
```

→ 97

```
s="WxYZ"
s=s.upper()
n=""
for i in s:
    if ord(i)<78:
        n+=chr(ord(i)+13)
    else:
        n+=chr(ord(i)+13+6)
print(n.upper())
```

→ JKLM

```
77+13
```

→ 90

```
s="abCDe"
s=s.upper()
n=""
for i in s:
    if ord(i)<78:
        n+=chr(ord(i)+13)
    else:
        n+=chr(ord(i)+13+6)
print(n.upper())
```

→ NOPQR

```
s="hello to this world"
l=s.split(" ")
ns=[]
for i in l:
    ns.append(i[0].upper()+i[1:-1]+i[-1].upper())
print(" ".join(ns))
```

→ Hello TO This World

```
s=input()
flag1=False
```

```

flag2=False
for i in s:
    if i.isalpha():
        flag1=True
    if i.isdigit():
        flag2=True
print(flag1 and flag2)

```

 56fhv7
 True

```

s=input()
vowels=set("aeiou")
l=set({})
for i in s:
    if i in vowels:
        l.add(i)
if len(l)==len(vowels):
    print("accepted")
else:
    print(" not accepted")

```

 seatuion
 accepted

```

#str1 = 'abcdef'
#str2 = 'defghia'
s1=input()
s2=input()
k=[]
c=0
for i in s1:
    if i in s2 and i not in k:
        k.append(i)
        c+=1
print(c)

```

 aabcddek1112@
 bb2211@55k
 5

```

s=input()
vowels=set("aeiou")
c=0
for i in s:
    if i in vowels:
        c+=1
print(c)

```

 geeksforgeeks
 5

```

arr=[2,3,4,5,0,6]
sum=0
for i in arr:
    sum+=i
print((len(arr)*(len(arr)+1))/2-sum)

```

 1

```

arr=[2,3,4,5,0,6]

res=0
for i in arr:
    res^=i
for j in range(len(arr)+1):
    res^=j
print(res)

```

 1

```
s=[1,1,2,1,3,2,1]
ans=0

for i in range(len(s)):
    sum=0
    for j in range(i,len(s)):
        sum+=j
        if sum==6:
            ans+=1
print(ans)
```

↔ 3

```
l=[3,3,9,2]
l.sort()
print(l)
```

↔ [2, 3, 3, 9]
None

```
l=[2,4,24,7]
print(sorted(l,reverse=True))
```

↔ [24, 7, 4, 2]

```
l.sort(reverse=True)
print(l)
```

↔ [24, 7, 4, 2]

```
l=['a','w','d']
l.sort()
print(l)
```

↔ ['a', 'd', 'w']

```
n=['aav','dkn','klk','aaasw']
n.sort()
print(n)
```

↔ ['aaasw', 'aav', 'dkn', 'klk']

Start coding or [generate](#) with AI.

▼ Tuples

```
t=()
print(type(t))
```

↔ <class 'tuple'>

```
i,j,k=('a','b','m')
print(i)
print(j)
print(k)
```

↔ a
b
m

```
a,b=(3,4,2,5,9)
print(a)
print(b)
```

```
-----  
ValueError                                Traceback (most recent call last)  
<ipython-input-3-c0f6ee4503bd> in <cell line: 1>()  
----> 1 a,b=(3,4,2,5,9)  
      2 print(a)  
      3 print(b)  
  
ValueError: too many values to unpack (expected 2)
```

Next steps: [Explain error](#)

```
a,*b=(24,5,3,"fv",'ffvb')  
print(a)  
print(b)
```

```
24  
[5, 3, 'fv', 'ffvb']
```

```
a,*b,c=(24,5,3,"fv",'ffvb')  
print(a)  
print(b)  
print(c)
```

```
24  
[5, 3, 'fv']  
ffvb
```

```
a,*b,c,d=(24,5,3,"fv",'ffvb')  
print(a)  
print(b)  
print(c)  
print(d)
```

```
24  
[5, 3]  
fv  
ffvb
```

```
a,*b,c,d*=(24,5,3,"fv",'ffvb') #multiple * operator not possible in unpacking  
print(a)  
print(b)  
print(c)  
print(d)
```

```
File "<ipython-input-8-a4d197163216>", line 1  
  a,*b,c,d*=(24,5,3,"fv",'ffvb')  
    ^  
SyntaxError: 'tuple' is an illegal expression for augmented assignment
```

Next steps: [Fix error](#)

#accessing element in tuples

```
t=tuple("ashika")  
for i in range(len(t)):  
    print(t[i])
```

```
a  
s  
h  
i  
k  
a
```

```
t[0]
```

```
'a'
```

```
t[-1]
```

```
'a'
```



```
t[-3]
```

```
↵ 'i'
```

```
#Slicing also same in list
```

```
t[2:4]
```

```
↵ ('h', 'i')
```

```
#tuple is immutable so can't add or del a specific element
```

```
t=(3,5,23,56,2)
```

```
t[1]=4
```

```
↵ -----
TypeError                                Traceback (most recent call last)
<ipython-input-14-0d3671ffa637> in <cell line: 4>()
      2
      3 t=(3,5,23,56,2)
----> 4 t[1]=4
```

```
TypeError: 'tuple' object does not support item assignment
```

Next steps:

[Explain error](#)

```
l=[1,2,100]
```

```
t=(3,5,23,56,2,1)
```

```
print(t)
```

```
↵ (3, 5, 23, 56, 2, [1, 2, 100])
```

```
l[0]=200
```

```
print(t)
```

```
↵ (3, 5, 23, 56, 2, [200, 2, 100])
```

```
t[5]=[1,3,4]
```

```
print(t)
```

```
↵ -----
TypeError                                Traceback (most recent call last)
<ipython-input-18-67681618fc44> in <cell line: 1>()
----> 1 t[5]=[1,3,4]
      2 print(t)
```

```
TypeError: 'tuple' object does not support item assignment
```

Next steps:

[Explain error](#)

```
l=[1,2,3,4]
```

```
m=l.copy()
```

```
l[0]=50
```

```
print(l)
```

```
print(m)
```

```
↵ [50, 2, 3, 4]
   [1, 2, 3, 4]
```

```
s=[5,3,0]
```

```
l=[1,2,3,4,s] #both got updated
```

```
m=l.copy()
```

```
s[0]=50
```

```
print(l)
```

```
print(m)
```

```
↵ [1, 2, 3, 4, [50, 3, 0]]
   [1, 2, 3, 4, [50, 3, 0]]
```

```
import copy
s=[5,3,0]
l=[1,2,3,4,s]
m=copy.deepcopy(l)
s[0]=50
print(l)
print(m)
```

```
[1, 2, 3, 4, [50, 3, 0]]
[1, 2, 3, 4, [5, 3, 0]]
```

#updating a tuple

```
t = ("python", "c", "c++")
t[1] = "Java"
```

#if needed to update convert it to list,update it then convert back to tuple

```
-----
TypeError                                Traceback (most recent call last)
<ipython-input-24-e78c0dd6f049> in <cell line: 5>()
      3 t = ("python", "c", "c++")
      4
----> 5 t[1] = "Java"
      6
      7 #if needed to update convert it to list,update it then convert back to tuple

TypeError: 'tuple' object does not support item assignment
```

Next steps: [Explain error](#)

```
l = [1,2,3,4,5]
del l[1:3]
print(l)
```

```
[1, 4, 5]
```

```
l = (1,2,3,4,5)
del l[1:3]
print(l)
```

```
-----
TypeError                                Traceback (most recent call last)
<ipython-input-26-4bf3e81e433d> in <cell line: 3>()
      1 l = (1,2,3,4,5)
      2
----> 3 del l[1:3]
      4 print(l)

TypeError: 'tuple' object does not support item deletion
```

Next steps: [Explain error](#)

```
l = (1,2,3,4,5)
del l
```

```
l
```

```
-----
NameError                                Traceback (most recent call last)
<ipython-input-28-cde25b5e10ad> in <cell line: 1>()
----> 1 l

NameError: name 'l' is not defined
```

Next steps: [Explain error](#)

```
l = ["8948279632687268", "9"]
l.sort()
print(l)
```

```
['8948279632687268', '9']
```

```
# sort

# sorted

t = (1,2,3,5,7,9,4,12,10)

t.sort() # will fail
```

```
-----
AttributeError                                Traceback (most recent call last)
<ipython-input-30-91be4252f0af> in <cell line: 7>()
      5 t = (1,2,3,5,7,9,4,12,10)
      6
----> 7 t.sort() # will fail

AttributeError: 'tuple' object has no attribute 'sort'
```

Next steps: [Explain error](#)

```
sorted(t) # always returns list object
```

```
[1, 2, 3, 4, 5, 7, 9, 10, 12]
```

```
#reverse
l=list("srujisha") # equivalent to l[::-1]
l.reverse()        # reverse in place
print(l)
```

```
['a', 'h', 's', 'i', 'j', 'u', 'r', 's']
```

```
list(reversed(l))
```

```
['s', 'r', 'u', 'j', 'i', 's', 'h', 'a']
```

```
t = tuple("Venkatesh")

t.reverse()
```

```
-----
AttributeError                                Traceback (most recent call last)
<ipython-input-34-4581c60decf7> in <cell line: 3>()
      1 t = tuple("Venkatesh")
      2
----> 3 t.reverse()

AttributeError: 'tuple' object has no attribute 'reverse'
```

Next steps: [Explain error](#)

```
tuple(reversed(t))
```

```
('h', 's', 'e', 't', 'a', 'k', 'n', 'e', 'V')
```

```
# count
```

```
l = [1,2,3,1,2,4,3,2,1,3,2,1,5,2,1,4,5]
```

```
l.count(1)
```

↗ 5

▼ sets

```
s={}
print(type(s)) #it will be not set,it is dict
```

↗ <class 'dict'>

```
#add
s={2,4,2,4,1,7,9}
s.add(5)
s
```

↗ {1, 2, 4, 5, 7, 9}

```
s={3,5,[2,3]}
s
```

↗

```
-----
TypeError                                Traceback (most recent call last)
<ipython-input-3-99916950c2a7> in <cell line: 1>()
----> 1 s={3,5,[2,3]}
      2 s

TypeError: unhashable type: 'list'
```

Next steps: [Explain error](#)

```
s={3,4,(2,0)}
s
```

↗ {(2, 0), 3, 4}

```
s={3,4,{3:2}}
s
```

↗

```
-----
TypeError                                Traceback (most recent call last)
<ipython-input-5-8628fdb0992f> in <cell line: 1>()
----> 1 s={3,4,{3:2}}
      2 s

TypeError: unhashable type: 'dict'
```

Next steps: [Explain error](#)

```
s={3,2,{0,9}}
s
```

↗

```
-----
TypeError                                Traceback (most recent call last)
<ipython-input-6-e6b776b85341> in <cell line: 1>()
----> 1 s={3,2,{0,9}}
      2 s

TypeError: unhashable type: 'set'
```

Next steps: [Explain error](#)

```
t=(2,3,2,1,1.0)
t
```

↗ (2, 3, 2, 1, 1.0)

```
s={1,1.0,0,False,True}
s
```

 {0, 1}

```
hash(1)
```

 1

```
hash((1,2))
```

 -3550055125485641917

```
hash({2,3})
```

 -----
Traceback (most recent call last)
[<ipython-input-11-ee149837e116>](#) in <cell line: 1>()
----> 1 hash({2,3})

TypeError: unhashable type: 'set'

Next steps: [Explain error](#)

```
hash([1,9])
```

 -----
Traceback (most recent call last)
[<ipython-input-12-16b4f43d49e3>](#) in <cell line: 1>()
----> 1 hash([1,9])

TypeError: unhashable type: 'list'

Next steps: [Explain error](#)

```
hash((2,3,4,[1]))
```

 -----
Traceback (most recent call last)
[<ipython-input-13-1c3375e737c7>](#) in <cell line: 1>()
----> 1 hash((2,3,4,[1]))

TypeError: unhashable type: 'list'

Next steps: [Explain error](#)

```
#accessing a set not possible..no indexing
s={2,3.4,9}
s[0]
```

 -----
Traceback (most recent call last)
[<ipython-input-14-8e969d517440>](#) in <cell line: 3>()
 1 #accessing a set not possible..no indexing
 2 s={2,3.4,9}
----> 3 s[0]

TypeError: 'set' object is not subscriptable

Next steps: [Explain error](#)

```
2 in s
```

 True

```
s={10,20,30}
s.add(22)
s
```

➞ {10, 20, 22, 30}

```
s.add(90,57,3)
s
```

➞ -----
TypeError Traceback (most recent call last)
 <ipython-input-18-e7fbf784291b> in <cell line: 1>()
----> 1 s.add(90,57,3)
 2 s

TypeError: set.add() takes exactly one argument (3 given)

Next steps: [Explain error](#)

```
s.add((90,57,3))
s
```

➞ {(90, 57, 3), 10, 20, 22, 30}

```
#update
s={10,20,30}
t={1,9,8}
s.update(t) #it will update s with al the elements added
s
```

➞ {1, 8, 9, 10, 20, 30}

```
s=s.update(t)
print(s)
```

➞ -----
AttributeError Traceback (most recent call last)
 <ipython-input-23-ce0fc68dd64a> in <cell line: 1>()
----> 1 s=s.update(t)
 2 print(s)

AttributeError: 'NoneType' object has no attribute 'update'

Next steps: [Explain error](#)

```
s={1,2,3}
t={5,6,7}
print(s.union(t)) #it will be stored as a separate set not storing in s itself
print(s)
```

➞ {1, 2, 3, 5, 6, 7}
{1, 2, 3}

```
a = {1,2,3}
b = [5,7,8, 3]
a.union(b) #in the bracket,it can be any iterable,but a should be a set
```

➞ {1, 2, 3, 5, 7, 8}

```
b.union(a)
```

➞ -----
AttributeError Traceback (most recent call last)
 <ipython-input-29-4878f99b7ef9> in <cell line: 1>()
----> 1 b.union(a)

AttributeError: 'list' object has no attribute 'union'

Next steps: [Explain error](#)

```
s={1,2,3}
s.update("RBR")
print(s)
```

```
↕ {1, 2, 3, 'R', 'B'}
```

```
s={1,2,3}
s.update(["RBR"])
print(s)
```

```
↕ {1, 2, 3, 'RBR'}
```

```
#remove
s={"abs","vkc","kkj","lty"}
s.remove("vkc")
s
```

```
↕ {'abs', 'kkj', 'lty'}
```

```
s.remove("sbjbd")  #it shows this key is not present error
s
```

```
↕ -----
      KeyError                                Traceback (most recent call last)
      <ipython-input-34-65338fac457f> in <cell line: 1>()
----> 1 s.remove("sbjbd")
      2 s

      KeyError: 'sbjbd'
```

Next steps: [Explain error](#)

```
#discard
s={"abs","vkc","kkj","lty"}
s.discard("abs")
s
```

```
↕ {'kkj', 'lty', 'vkc'}
```

```
s.discard("hgjwdb")  #no error generated if element to be removed is not present
s
```

```
↕ {'kkj', 'lty', 'vkc'}
```

```
#pop
#no indexing,therefore it remove any 1 item randomly
s={"abs","vkc","kkj","lty"}
print(s.pop())
print(s)
```

```
↕ kkj
   {'lty', 'vkc', 'abs'}
```

```
#set operations
```

```
a = {1,2,3}
b = {4,5,6}
c = {7,8,9}
d = {10,11,12}
e = {14,15,16}
f = [17,18,19]
g = ["Venky"]

a.union(b,c,d,e,f,g)
```

```
➦ {1, 10, 11, 12, 14, 15, 16, 17, 18, 19, 2, 3, 4, 5, 6, 7, 8, 9, 'Venky'}
```

```
# union also represented by |  
a|b  
print(a)
```

```
➦ {1, 2, 3}
```

```
a|e #only on sets
```

```
➦ -----  
TypeError                                Traceback (most recent call last)  
  <ipython-input-44-559b492ab00b> in <cell line: 1>()  
    ----> 1 a|e  
  
TypeError: unsupported operand type(s) for |: 'set' and 'tuple'
```

Next steps: [Explain error](#)

```
#union update  
a|=b  
print(a) #here a is also update with the union value
```

```
➦ {1, 2, 3, 4, 5, 6}
```

```
#intersection  
a = {1,2,3,4}  
b = {2,3,5,7}  
print(a.intersection(b))  
print(a)
```

```
➦ {2, 3}  
{1, 2, 3, 4}
```

```
a&b
```

```
➦ {2, 3}
```

```
a&=b  
print(a) #intersectionupdate
```

```
➦ {2, 3}
```

```
# difference  
  
a = {1,2,3,4}  
b = {2,3,5,7}  
  
a.difference(b)
```

```
➦ {1, 4}
```

```
a-=b  
print(a)
```

```
➦ {1, 4}
```

```
# symmetric_difference #common in both is discarded  
  
gate_da = {"ML", "Python", "LA", "Calculus", "Aptitude"}  
  
gate_cs = {"OS", "C++", "LA", "Calculus", "Aptitude"}  
  
gate_da.symmetric_difference(gate_cs)
```

```
➦ {'C++', 'ML', 'OS', 'Python'}
```

```
# ^  
gate_da ^ gate_cs
```



```
➦ {'C++', 'ML', 'OS', 'Python'}
```

```
gate_da.symmetric_difference_update(gate_cs)

print(gate_da)
print(gate_cs)
```

```
➦ {'C++', 'ML', 'Python', 'OS'}
{'Calculus', 'LA', 'C++', 'Aptitude', 'OS'}
```

```
a = {1,2,3,4,5,6}

a.clear()
```

```
a = {1,2,3,4,5,6}

del a
```

```
# isdisjoint

a = {1,2,3,4}
b = {2,3,5,7}

a.isdisjoint(b) #boolean value-if nothing in common
```

```
➦ False
```

```
# issuperset

a = {1,2,3,4}
b = {2,3,5,7}

a.issuperset(b)
```

```
➦ False
```

```
#subset

a = {1,2,3,5}
b = {1,2}

b.issubset(a)
```

```
➦ True
```

frozen sets

```
f = frozenset([10,20,30,1,True,0,False])
print(f)
print(type(f)) #frozen sets are immutable
```

```
➦ frozenset({0, 1, 10, 20, 30})
<class 'frozenset'>
```

```
f.add(12)
```

```
➦ -----
AttributeError                                Traceback (most recent call last)
<ipython-input-67-a6783dda8516> in <cell line: 1>()
----> 1 f.add(12)
```

AttributeError: 'frozenset' object has no attribute 'add'

Next steps: [Explain error](#)

```
f.remove(10)
```



```
-----
AttributeError                                Traceback (most recent call last)
<ipython-input-68-09e07bffb3eb> in <cell line: 1>()
----> 1 f.remove(10)

AttributeError: 'frozenset' object has no attribute 'remove'
```

Next steps:

[Explain error](#)

```
# UNION
# INTERSECTION
# DIFFERENCE
# SYMMETRIC_DIFFERENCE
# COPY
#same as in sets
```

Dictionary

```
#key value pair  {}
#initialise
d={}
type(d)
```



```
dict
```

```
d={'a':2,'s':3}
print(d)
```



```
{'a': 2, 's': 3}
```

```
d=dict()
print(d)
print(type(d))
```



```
{ }
<class 'dict'>
```

```
#dict is ordered
d={'a':2,'s':3,'w':34,1:[2,4,5]}
print(d)
```



```
{'a': 2, 's': 3, 'w': 34, 1: [2, 4, 5]}
```

```
d[0] #not indexed by sequence,indexed by keys
```



```
-----
KeyError                                Traceback (most recent call last)
<ipython-input-43-123a9cc6df61> in <cell line: 1>()
----> 1 d[0]

KeyError: 0
```

Next steps:

[Explain error](#)

```
d['a']
```



```
2
```

```
d={'a':9,'b':2,'d':6}
for i in d:
    print(i)
```



```
a
b
```

d

```
1.0==1
```

```
➦ True
```

```
d={1:'a',1.0:'bf',1.00:'g',0:"adv",True:"al",False:"sdd"}
print(d) #it takes others as duplicates
```

```
➦ {1: 'a', 0: 'sdd'}
```

```
d={1:"sd",True:8}
print(d) #here the value is also updated
```

```
➦ {1: 8}
```

```
#duplicate values are allowed,not duplicate keys
#print in same but indexing using keys only
```

```
d={'a':9,'b':2,'d':6}
for i in d:
    print(i)
    print(d[i])
```

```
➦ a
  9
  b
  2
  d
  6
```

```
d={'a':10,'f':20,'a':100}
d #since dup key not allowed,so its value is updated by last value of that key
```

```
➦ {'a': 100, 'f': 20}
```

```
a="ashika"
s={a:12}
s
```

```
➦ {'ashika': 12}
```

```
#dict method
d=dict(a=8, e=9, 3=s0 ,1=k)
print(d)
```

```
➦ File "<ipython-input-77-e05ac4df362d>", line 2
  d=dict(a=8, e=9, 3=s0 ,1=k)
                    ^
SyntaxError: expression cannot contain assignment, perhaps you meant "=="?
```

Next steps: [Fix error](#)

```
#dict method
d=dict(a=8, e=9,1=k)
print(d)
```

```
➦ File "<ipython-input-79-f7a98f0f42b8>", line 2
  d=dict(a=8, e=9,1=k)
                ^
SyntaxError: expression cannot contain assignment, perhaps you meant "=="?
```

Next steps: [Fix error](#)

```
#dict method
d=dict(a=8, e=9,s=3,k=9)
print(d)
```

```
➦ {'a': 8, 'e': 9, 's': 3, 'k': 9}
```

```
#dict method
d=dict(a=8, e=9, "3"="0" , "1"="k")
print(d)
```

File "[<ipython-input-83-ffc3f4f5aaaf>](#)", line 2
d=dict(a=8, e=9, "3"="0" , "1"="k")
 ^
SyntaxError: expression cannot contain assignment, perhaps you meant "=="?

Next steps: [Fix error](#)

```
d1={'s':10,'d':30,'l':90}
d2=d1
print(id(d1))
print(id(d2))
```

140144982583104
140144982583104

```
d1['s']=2000
print(d1)
print(d2) #copy also affected
```

{'s': 2000, 'd': 30, 'l': 90}
{'s': 2000, 'd': 30, 'l': 90}

```
#use copy()
d3=d1.copy() #or use dict(d1)-same effect but as told in tuple
d1['s']=0 #it will affect the copy also if a list inside it is modified
print(d1)
print(d3)
```

{'s': 0, 'd': 30, 'l': 90}
{'s': 2000, 'd': 30, 'l': 90}

```
s=dict(("a",4),("d",0),(100,"p"))
print(s)
```

TypeError Traceback (most recent call last)
[<ipython-input-1-eedf3797753b>](#) in <cell line: 1>()
----> 1 s=dict(("a",4),("d",0),(100,"p"))
 2 print(s)

TypeError: dict expected at most 1 argument, got 3

Next steps: [Explain error](#)

```
s=dict(((("a",4),("d",0),(100,"p"))))
print(s)
```

{'a': 4, 'd': 0, 100: 'p'}

```
s=dict([(("a",4),("d",0),(100,"p"))])
print(s)
```

{'a': 4, 'd': 0, 100: 'p'}

```
s=dict([(("a",4),("d",0),(100,"p",0))])
print(s)
```

ValueError Traceback (most recent call last)
[<ipython-input-4-090c2ca2ed46>](#) in <cell line: 1>()
----> 1 s=dict([(("a",4),("d",0),(100,"p",0))])
 2 print(s)

ValueError: dictionary update sequence element #2 has length 3; 2 is required

Next steps: [Explain error](#)

```
#can add also keyword argument with this
s=dict(((("a",4),("d",0),(100,"p")),virat=120))
print(s)
```

```
File "<ipython-input-6-e5feb97db81d>", line 2
    s=dict(((("a",4),("d",0),(100,"p")),virat=120))
                                ^
```

SyntaxError: invalid syntax

Next steps: [Fix error](#)

```
s=dict([(("a",4),("d",0),(100,"p")),virat=100,a=90])
print(s)
```

```
{'a': 90, 'd': 0, 100: 'p', 'virat': 100}
```

```
#creating dictionary from fromkeys method
s=["d","f",34,'p']
d=dict.fromkeys(s) #by default assigned with none as value
d
```

```
{'d': None, 'f': None, 34: None, 'p': None}
```

```
#give value also as parameter
d=dict.fromkeys(s,0)
d
```

```
{'d': 0, 'f': 0, 34: 0, 'p': 0}
```

```
d=dict.fromkeys(s,[1,32,4])
d
```

```
{'d': [1, 32, 4], 'f': [1, 32, 4], 34: [1, 32, 4], 'p': [1, 32, 4]}
```

#keys and value in dictionary

```
#nested dictionary
d={'a':[1,3,4], 'b':(3,0,4), 'c':{33,90,60}, 'd':{4:60, 's':70}}
d
```

```
{'a': [1, 3, 4], 'b': (3, 0, 4), 'c': {33, 60, 90}, 'd': {4: 60, 's': 70}}
```

```
d={(12,3,4):'a'} #tuple as key
d
```

```
{(12, 3, 4): 'a'}
```

```
d={[12,3,4]:'a'} #list as key not possible
d
```

```
-----
TypeError                                 Traceback (most recent call last)
<ipython-input-15-bdd501dd5c91> in <cell line: 1>()
----> 1 d={[12,3,4]:'a'} #tuple as key
      2 d
```

TypeError: unhashable type: 'list'

Next steps: [Explain error](#)

#list,set,dict cannot be given as keys since they are mutable

```
d={(2,3,4,[89]):'s'}
d
```

```
-----
TypeError                                Traceback (most recent call last)
<ipython-input-18-0c9c79b85601> in <cell line: 1>()
----> 1 d={(2,3,4,[89]):'s'}
      2 d

TypeError: unhashable type: 'list'
```

Next steps: [Explain error](#)

```
d={'s':10,'d':30,'l':90}
len(d)    #no of keys
```

```
3
```

```
d={'s':10,'d':30,'l':90,1:9,1.0:'e'}
len(d)
```

```
4
```

```
#keys,values,items
d={'virat':100,'rohit':90,'sehwag':80}
print(d.keys())
print(d.values())
print(d.items())
```

```
dict_keys(['virat', 'rohit', 'sehwag'])
dict_values([100, 90, 80])
dict_items([('virat', 100), ('rohit', 90), ('sehwag', 80)])
```

```
for k in d.values():
    print(k)
```

```
100
90
80
```

```
for k in d.keys():
    print(k)
```

```
virat
rohit
sehwag
```

```
for k in d.items():
    print(k)
```

```
('virat', 100)
('rohit', 90)
('sehwag', 80)
```

```
for k,v in d.items():
    print(k,v)
```

```
virat 100
rohit 90
sehwag 80
```

```
#get() method
d={'virat':100,'rohit':90,'sehwag':80}
d.get('virat')
```

```
100
```

```
a=d.get('ashika')    #since not in dict returns none value
print(a)
```

```
None
```

```
#can give default value to get also,if not present it will be shown
a=d.get('ashika',-1)
```

```
print(a)
```

```
➞ -1
```

```
a=d.get('ashika',"key not found")
print(a)
```

```
➞ key not found
```

```
d.get('virat',"key not found")
```

```
➞ 100
```

```
#modifying dict
d={'virat':100,'rohit':90,'sehwag':80}
d['virat']=[2,3,4]
print(d)
```

```
➞ {'virat': [2, 3, 4], 'rohit': 90, 'sehwag': 80}
```

```
#in operator
'virat' in d #it will check in keys not value
```

```
➞ True
```

```
#if want to check in values
90 in d.values()
```

```
➞ True
```

```
#update method
d={'name':'ashi','age':22,'profession':'teaching'}
d
```

```
➞ {'name': 'ashi', 'age': 22, 'profession': 'teaching'}
```

```
d.update(name='ashika')
d
```

```
➞ {'name': 'ashika', 'age': 22, 'profession': 'teaching'}
```

```
d.update(name='aaashi',profession='lecturing')
d
```

```
➞ {'name': 'aaashi', 'age': 22, 'profession': 'lecturing'}
```

```
d={'name':'ashi','age':22,'profession':'teaching'}
s={'salary':100000000,'education':'Mtech'}
d.update(s)
d
```

```
➞ {'name': 'ashi',
    'age': 22,
    'profession': 'teaching',
    'salary': 100000000,
    'education': 'Mtech'}
```

```
d={'name':'ashi','age':22,'profession':'teaching'}
s=(("salary",1000000000),('education','Mtech'))
d.update(s)
d
```

```
➞ {'name': 'ashi',
    'age': 22,
    'profession': 'teaching',
    'salary': 1000000000,
    'education': 'Mtech'}
```

```
#pop()
d.pop()
```

```

-----
TypeError                                Traceback (most recent call last)
<ipython-input-7-4419b6980efa> in <cell line: 2>()
      1 #pop()
----> 2 d.pop()

TypeError: pop expected at least 1 argument, got 0
-----

```

Next steps: [Explain error](#)

```
d.pop('age')
```

```
↵ 22
```

```
d
```

```
↵ {'name': 'ashi',
   'profession': 'teaching',
   'salary': 10000000000,
   'education': 'Mtech'}
```

```
#popitem()
print(d)
print(d.popitem()) #its return type is tuple
print(d)
```

```
↵ {'name': 'ashi', 'profession': 'teaching', 'salary': 10000000000, 'education': 'Mtech'}
   ('education', 'Mtech')
   {'name': 'ashi', 'profession': 'teaching', 'salary': 10000000000}
```

```
del d["name"]
```

```
d
```

```
↵ {'profession': 'teaching', 'salary': 10000000000}
```

```
d.clear()
```

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.